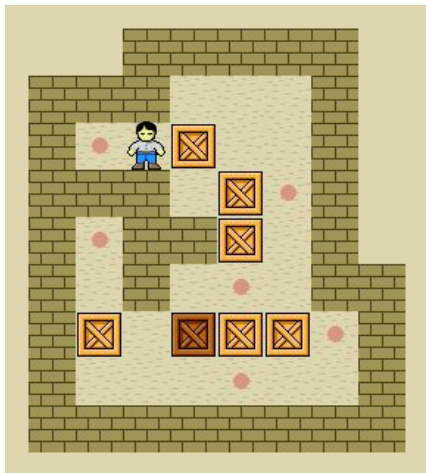


The impact of memory strategies on LLM-based Sokoban gameplay



CS348K Final Presentation
Aditri Patil and Kerui Lu

1:30 - Aditri Part 1

1:30-4:30 Workflow

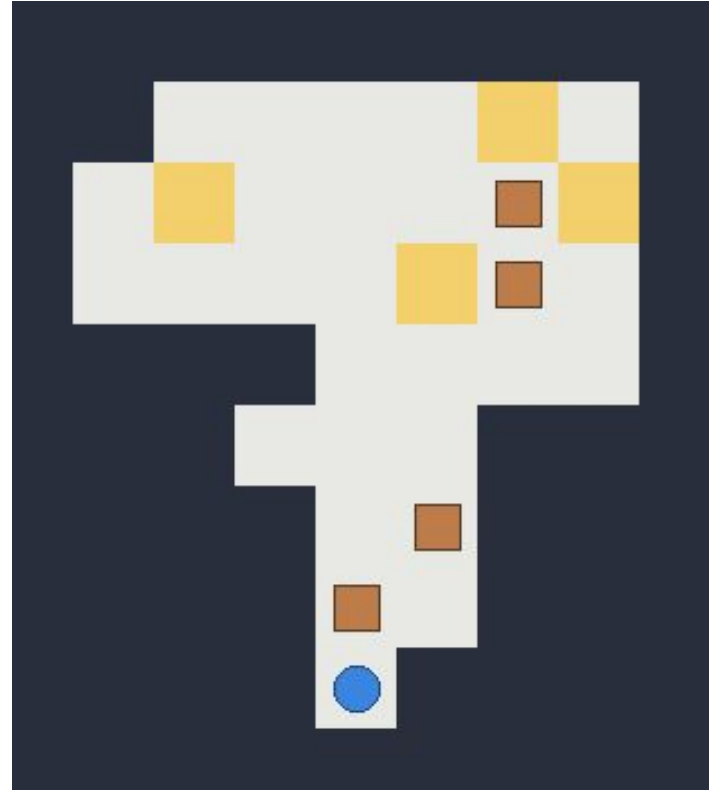
4:30-6 same level

6-7:15 train to eval

7:15-8:00 Next steps +
conclusions

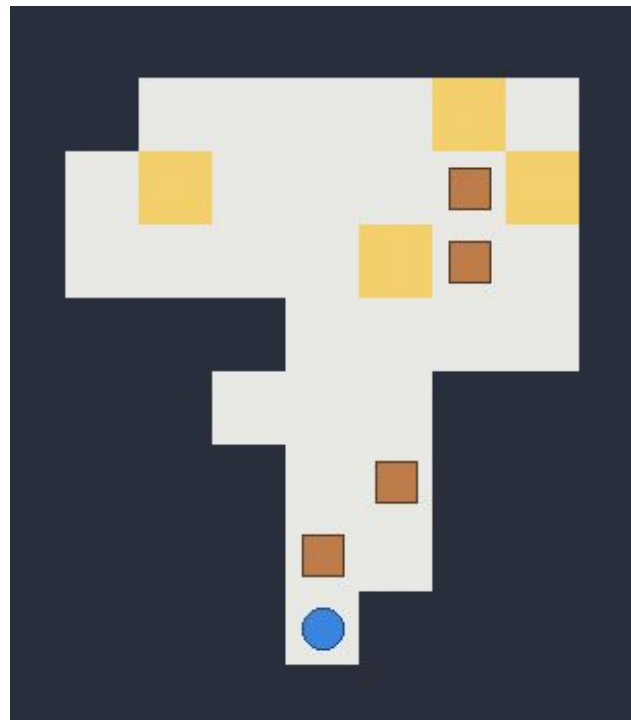
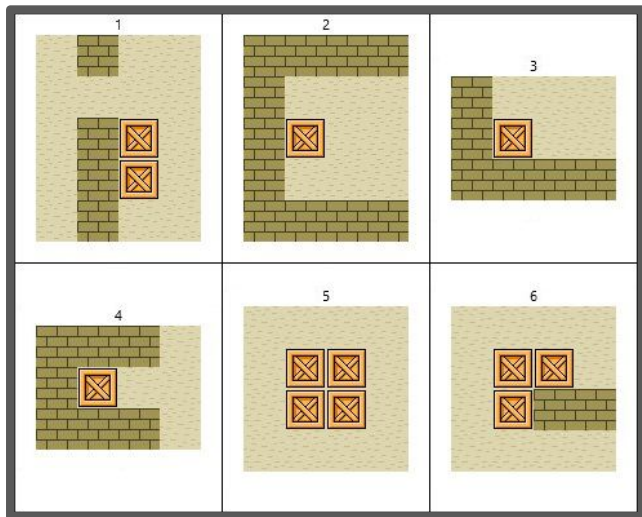
What is Sokoban?

Goal: Push all boxes
onto target square



Why is Sokoban hard for LLMs?

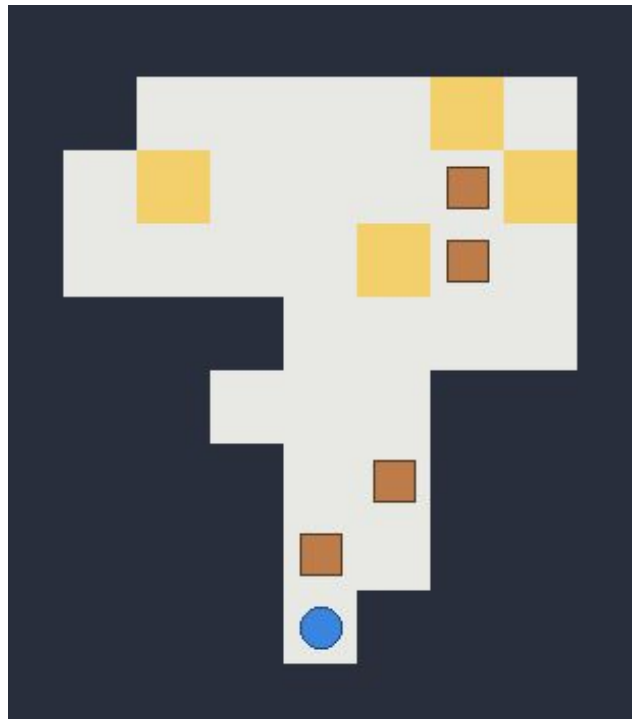
1. Irreversible moves → deadlocks



Goal: Push all boxes onto target square

Why is Sokoban hard for LLMs?

1. Irreversible moves → deadlocks
2. Coordinate tracking of multiple boxes

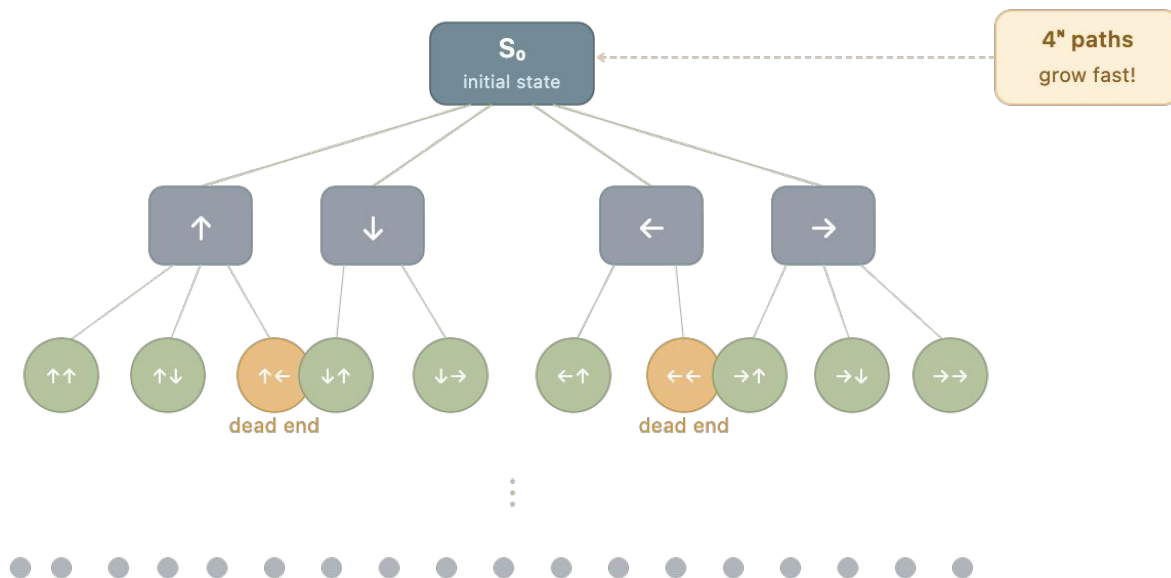


Goal: Push all boxes onto target square

Why is Sokoban hard for LLMs?

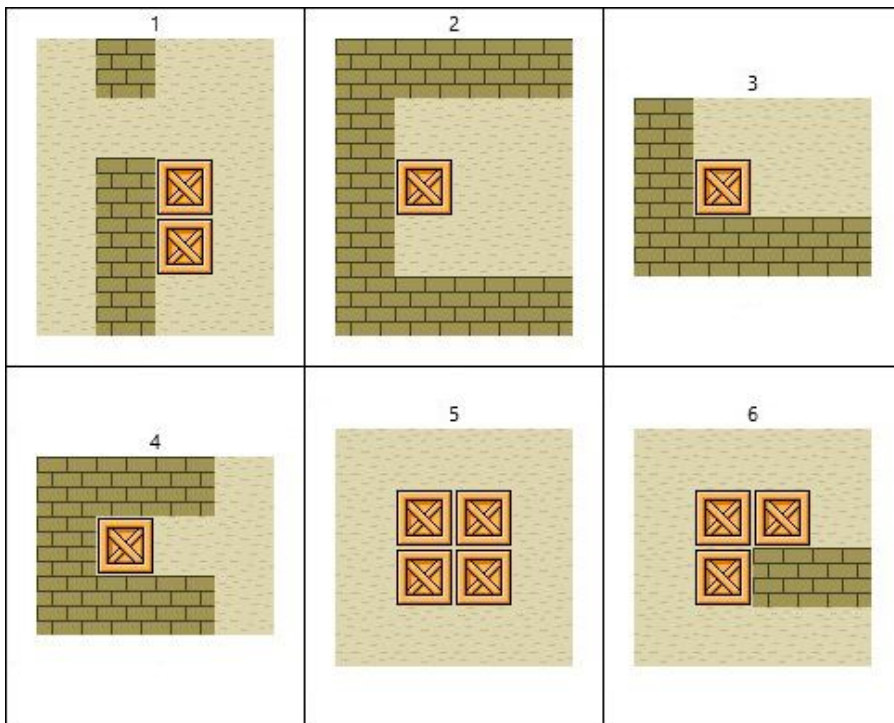
1. Irreversible moves → deadlocks
2. Coordinate tracking of multiple boxes

3. Long Horizon Planning



Challenges of LLM Sokoban gameplay

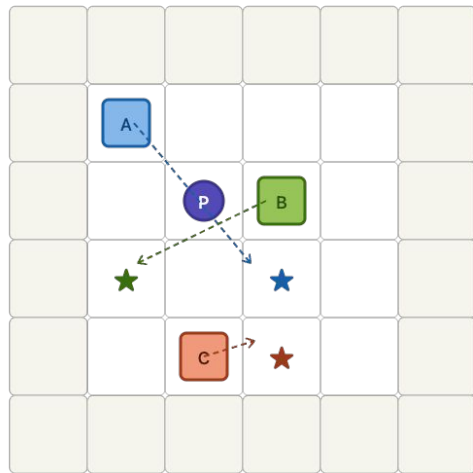
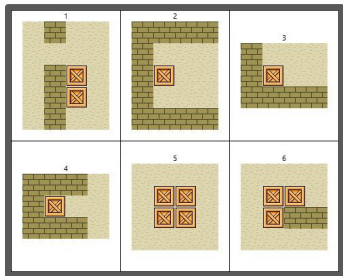
1. Irreversible moves → deadlocks



Challenges of LLM Sokoban gameplay

2. Coordinate tracking of multiple boxes

1. Irreversible moves → deadlocks



The player moves one step at a time

What the LLM must track

Player  (row 2, col 2)

Box A  (row 1, col 1)

Box B  (row 2, col 3)

Box C  (row 4, col 2)

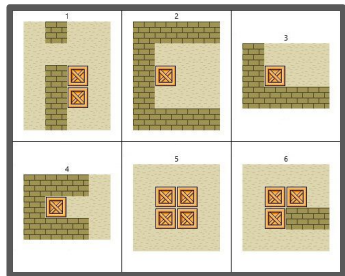
Targets  A→(3,3)

 B→(3,1) C→(4,3)

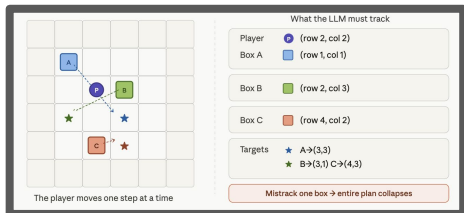
Mistrack one box → entire plan collapses

Challenges of LLM Sokoban gameplay

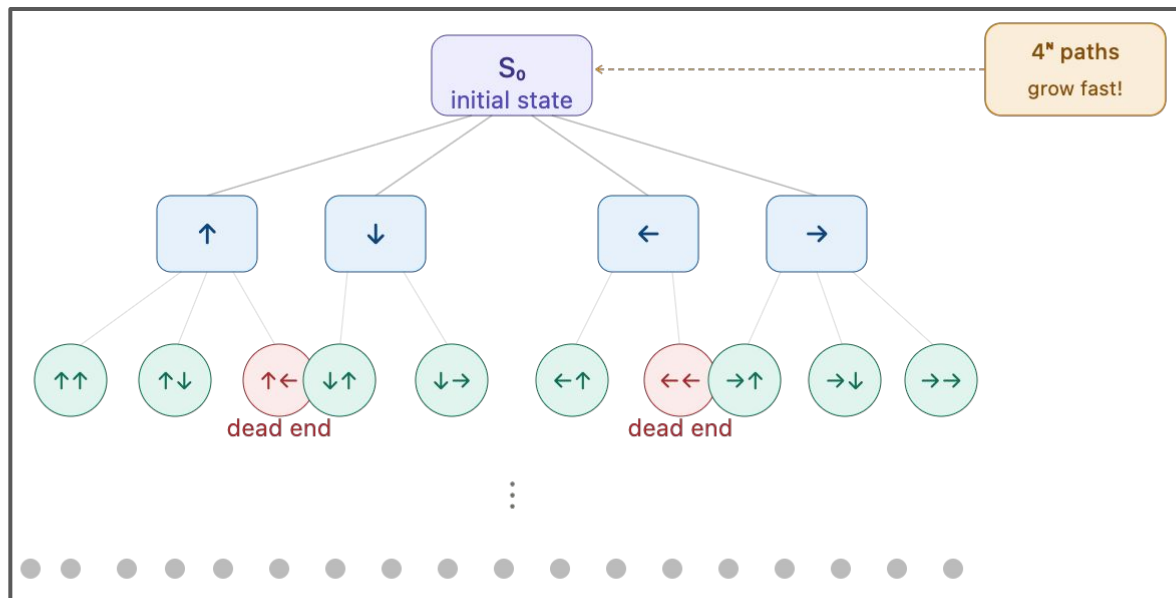
1. Irreversible moves → deadlocks



2. Coordinate tracking of multiple boxes

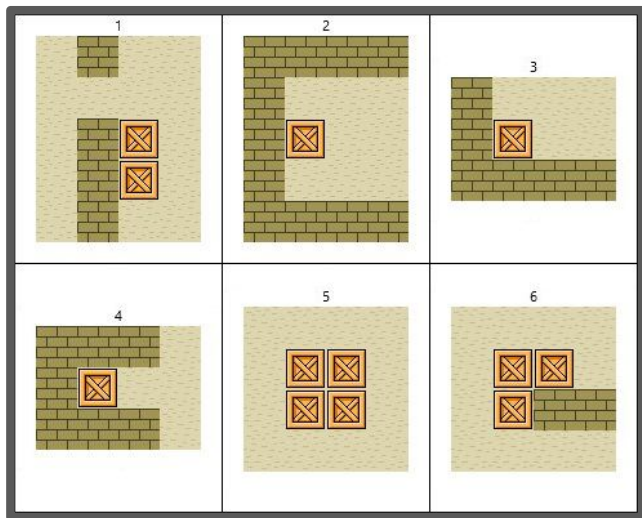


3. Long Horizon Planning

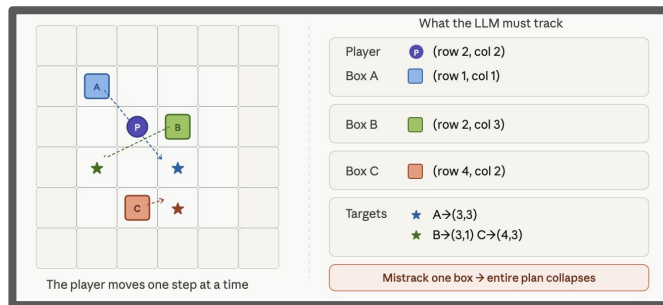


Why Sokoban?

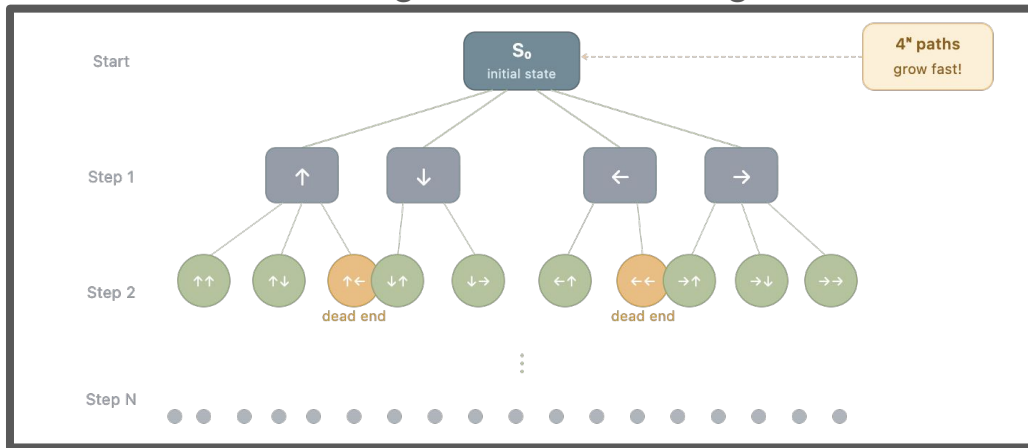
1. Irreversible moves → deadlocks



2. Coordinate tracking of multiple boxes



3. Long Horizon Planning



Our Baselines

Our prompt baselines

12 train levels, no memory, temp=0, verifier checked

Prompt / scaffold	Solved	Takeaway
Coordinate push plan	2/12	Best simple baseline
Explicit legality self-check	1/12	More rules did not help
Structured state trace fields	2/12	No valid-rate gain
Indexed boxes + compact trace	1/12	Less coordinate burden
Verifier-guided repair attempt	1/12	Repair did not recover
Rules + push legality examples	1/12	Better grounding, same result

Initial Research

Our Baselines

Our prompt baselines

12 train levels, no memory, temp=0, verifier checked

Prompt / scaffold	Solved	Takeaway
Coordinate push plan	2/12	Best simple baseline
Explicit legality self-check	1/12	More rules did not help
Structured state trace fields	2/12	No valid-rate gain
Indexed boxes + compact trace	1/12	Less coordinate burden
Verifier-guided repair attempt	1/12	Repair did not recover
Rules + push legality examples	1/12	Better grounding, same result

Prior Benchmarks

Model	Harness	Sokoban
claude-3-7-sonnet-20250219 (thinking)	No Yes	0.0±0.0 2.3±1.5
deepseek-r1	No Yes	N/A 1.3±1.2
gemini-2.5-pro-preview-05-06 (thinking)	No Yes	1.0±0.0 4.3±0.6
o3-2025-04-16 *	No Yes	2.0±0.0 8.0±2.8
o4-mini-2025-04-16	No Yes	1.3±0.6 5.3±1.2
Random	—	0.0±0.0

Source: Table 1 (Truncated) in [LMGame](#) Bench: How Good are LLMs at Playing Games? By Zhang et al

Why did we think of memory? Specifically raw and heuristic?

Prior research:

- LMGame: applies visual and memory module Sokoban. Does not compare which is better
- Reflexion: reinforce language agents through linguistic feedback on coding benchmark
- Synapse: Uses retrieved full trajectories as in-context examples for long-horizon computer-control tasks.

Can memory of previous gameplay trajectories help improve LLM
Sokoban performance?



What kinds of memory help
improve LLM Sokoban
performance

What kinds of memory help improve LLM Sokoban performance?

4 memory strategies

- Non memory – no prior experience
- Raw trajectory memory – storing failed attempts
- Compact Summary
- Reflection heuristic – abstract learned rules from failures

How to pick which ones? What parts?

How to generate these? What might be helpful?

Same-level Memory Retry vs Train-to-Eval Heuristic

What kinds of memory help improve LLM Sokoban performance?

No memory

Prompt (rules,
coordinates, task)

Raw memory

Prompt (rules,
coordinates, task)

model_plan

Steps before failure

Failure analysis

Board snapshots

Compact

Prompt (rules,
coordinates, task)

Failure subtype

Failure reason

LLM-generated Heuristics

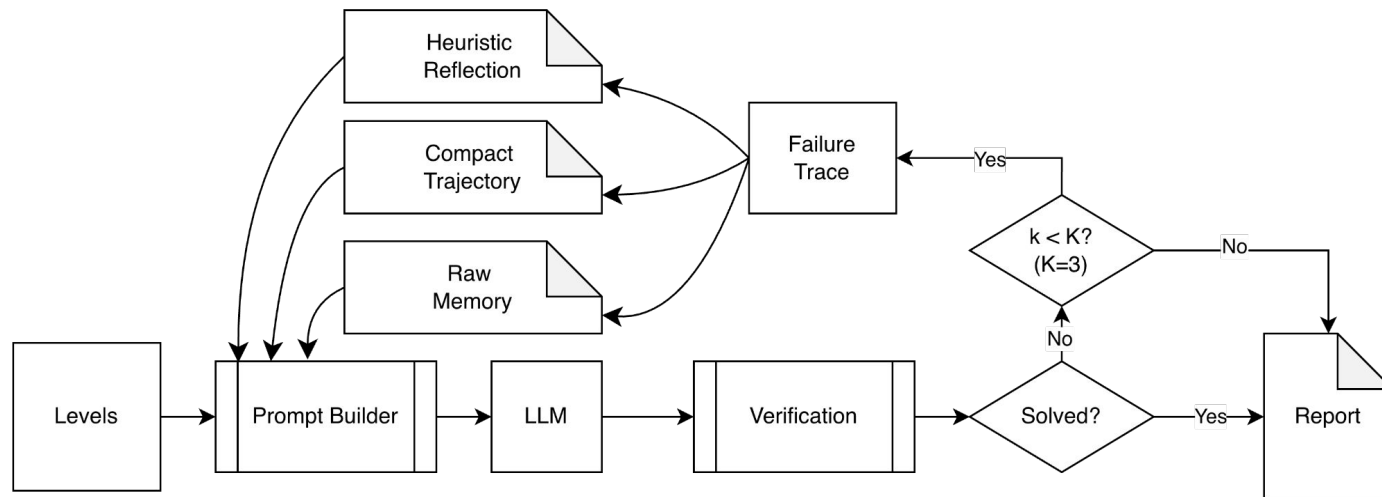
Prompt (rules,
coordinates, task)

LLM-generated
heuristics based
on memory
trajectories

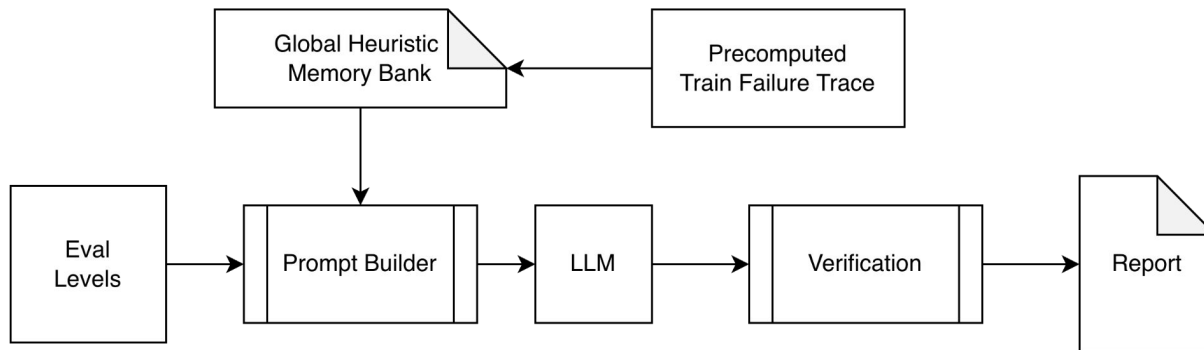
```
def build_reflection_prompt(raw_memory: RawTrajectoryMemory, config: MemoryRenderConfig) -> str:
    return (
        "You are analyzing failed Sokoban trajectories.\n"
        f"Prompt version: {REFLECTION_PROMPT_VERSION}\n\n"
        "Goal: distill concise heuristic rules that could help a future full-path push planner avoid similar failures.\n"
        "The future planner outputs JSON push intents such as {\"box\": [3, 4], \"push\": \"Right\"}; a local verifier expands reachable walking paths.\n"
        "Use prescriptive rules. Do not replay the trajectories.\n"
        "Return a JSON array of strings only.\n\n"
        f"Failed trajectory memory:\n{raw_memory.render(config)}"
    )
```

Our System

Same-Level Memory Retry Pipeline

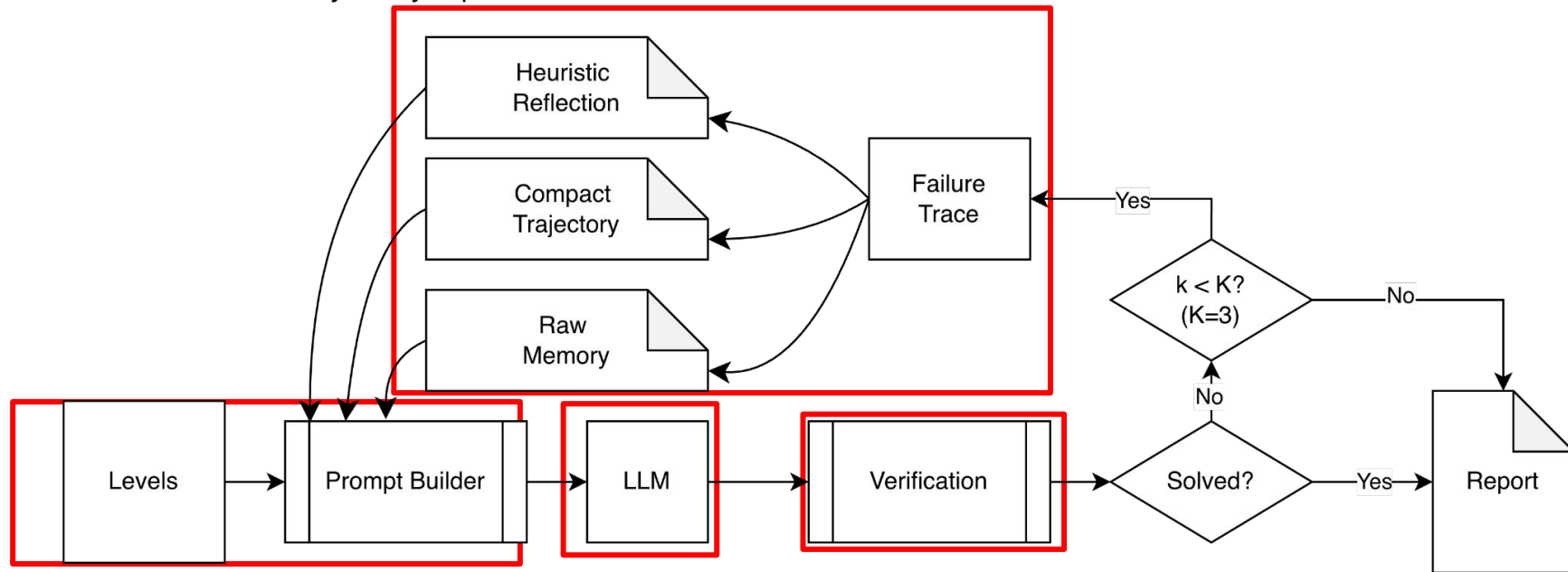


Train-to-Eval Heuristic Pipeline



Challenge of Our System: Harness Engineering

Same-Level Memory Retry Pipeline



Challenge #1: Help LLMs understand Sokoban



```
"Board symbols:",  
"# wall",  
"space empty floor",  
". target",  
"$ box",  
"* box on target",  
"@ player",  
"+ player on target",
```

Grid Representation

```
#####  
# .@#####  
# $ $ #####  
# #####  
# #####  
# #####  
# #####  
# $ . #####  
# . $ #####  
#####
```



google-deepmind / boxoban-levels

<https://github.com/google-deepmind/boxoban-levels>



Stratum	Avg wall density	Avg reachable ratio	Avg initial legal pushes	Avg difficulty score
open	0.618	0.892	10.38	0.306
middle	0.682	0.577	6.62	0.482
constrained	0.690	0.063	1.25	0.790

Each difficult stratum contains 16 levels.

Each level is a 10x10 Sokoban puzzles with 4 boxes and 4 targets.

Level Selection

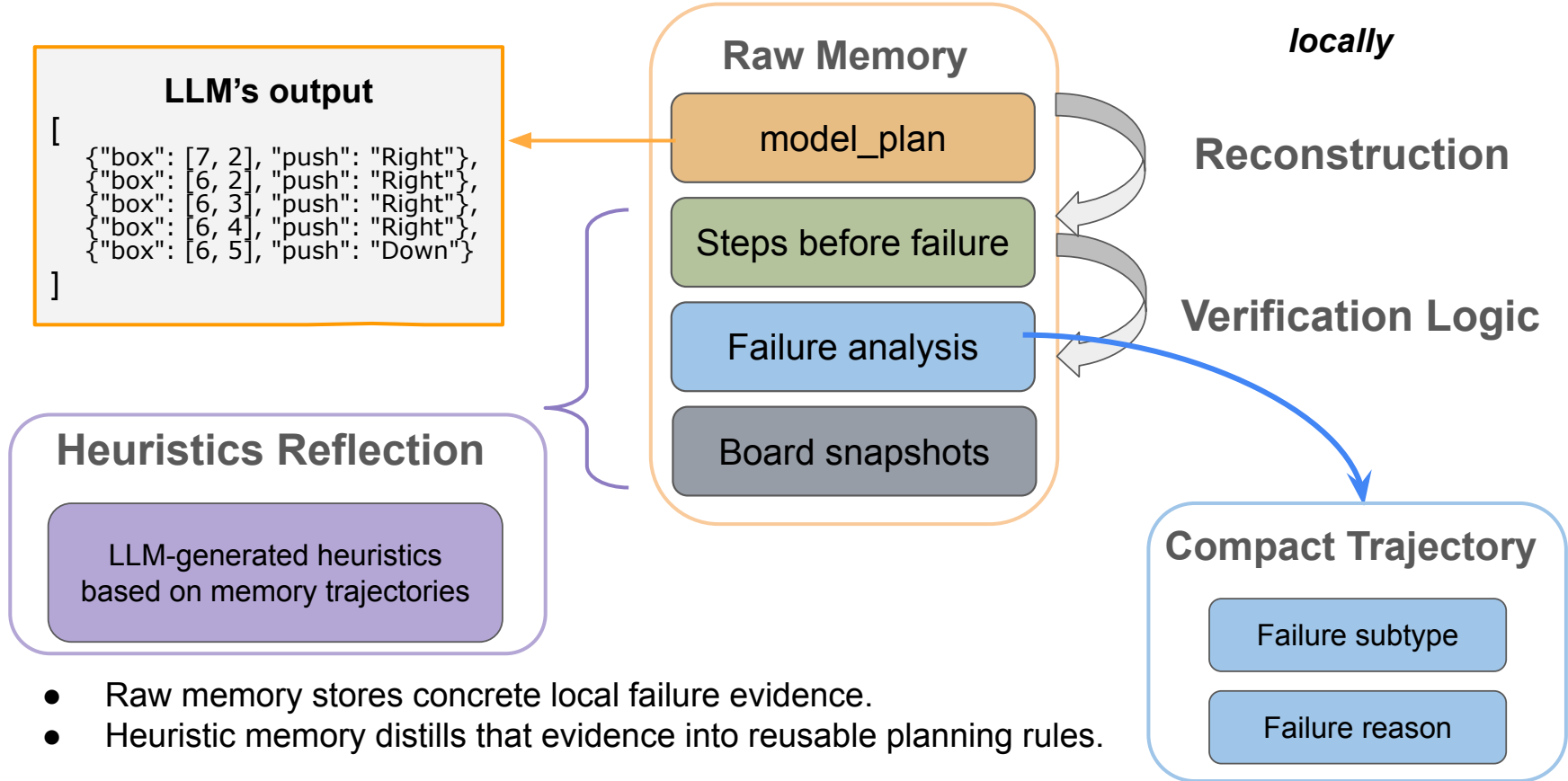
Challenge #2: Achieve Reliable LLM Solving Rate

Model	Reasoning effort	Max output tokens	Solve rate
gpt-5-nano	none	512	2/12 = 16.7%
gpt-5.2	low	8192	5/12 = 41.7%
gpt-5.2	low	16384	8/24 = 33.3%
gpt-5.2	medium	32768	13/24 = 54.2%

Goal: Select a configuration that has some baseline capability but still leaves room for improvement.

Definition of Solved: All boxes are on target positions, and no invalid steps occur during the process.

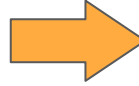
Challenge #3: Design Memory Conditions



Challenge #3: Memory Creation

LLM's output

```
[  
  {"box": [7, 2], "push": "Right"},  
  {"box": [6, 2], "push": "Right"},  
  {"box": [6, 3], "push": "Right"},  
  {"box": [6, 4], "push": "Right"},  
  {"box": [6, 5], "push": "Down"}  
]
```



A Raw Memory Evidence Item

```
{  
  "level_id": "boxoban_unfiltered_train_000_130",  
  "status": "invalid_plan",  
  
  "model_plan": [  
    {"box": [7, 2], "push": "Right"},  
    {"box": [6, 2], "push": "Right"},  
    {"box": [6, 3], "push": "Right"},  
    {"box": [6, 4], "push": "Right"},  
    {"box": [6, 5], "push": "Down"}  
  ],  
  
  "executed_steps_before_failure": [  
    {  
      "step": 0  
      ...  
      "step": 20,  
      "action": "Right",  
      "planned_push": {"box": [6, 4], "push": "Right"},  
      "pushed_box": true,  
      "next_state": "board after pushing box from [6,4] to [6,5]"  
    }  
  ],  
  
  "failed_push": {  
    "failure_push_index": 4,  
    "intent": {"box": [6, 5], "push": "Down"},  
    "required_standing_cell": [5, 5],  
    "destination_cell": [7, 5],  
    "player_reachable_to_standing": false,  
    "failure_reason": "required_push_position_unreachable"  
  },  
  
  "board_snapshots": {  
    "initial_board": "...",  
    "board_before_failed_push": "...",  
    "board_after_last_successful_push": "..."  
  }  
}
```

LLM generates plan only



local parser parses plan



local executor/verifier executes plan



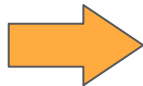
local env records trajectory



local compression creates raw memory record

Challenge #3: Memory Creation

- Model Plan
 - LLM-proposed push sequence
- Local Execution
- Failure Diagnosis
 - Verifier identifies why the next push is invalid
- Board Snapshots
 - Initial, pre-failure, and last successful states



Raw failure evidence



LLM reflection prompt summarizes repeated failure patterns



local scope classifier filters heuristic



store accepted heuristic memory



render heuristic rules in later prompts

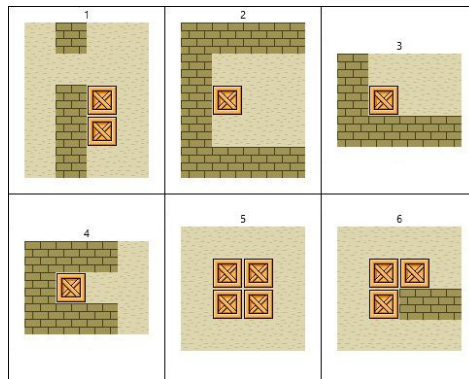
A Heuristic Memory Item

```
{  
  "memory_type": "heuristic_reflection",  
  "source": "raw failure trajectories",  
  "scope": "global_allowed",  
  
  "heuristic": "Before emitting a push intent,  
  ensure the required standing cell behind the box  
  is reachable in the current state; if not, first plan  
  pushes that open access or abandon that push  
  direction.",  
  
  "used_in_prompt_as": "reflection heuristic rule"  
}
```

- Raw memory stores concrete local failure evidence.
- Heuristic memory distills that evidence into reusable planning rules.

Challenge #4: Design the Verifier

Stage	Local check
Parse	JSON array of push intents
Resolve box	target box exists now
Push geometry	destination cell is free
Standing cell	player standing cell is physically open
Reachability	BFS path from player to standing cell
Execute	local env walks to push position, then pushes
Terminal check	solved / deadlocked / unfinished



Deadlock Check:

1. Non-target corner
2. Wall segment with no target / no exit
3. 2x2 freeze
4. Two-box freeze

Failure Type

- **Invalid_plan:** The proposed push cannot be legally executed.
- **Deadlock:** A push was executed, but the resulting board is locally detected as deadlocked.
- **Plan_exhausted:** All planned pushes were executed, but the puzzle is not solved.

Our Hypotheses

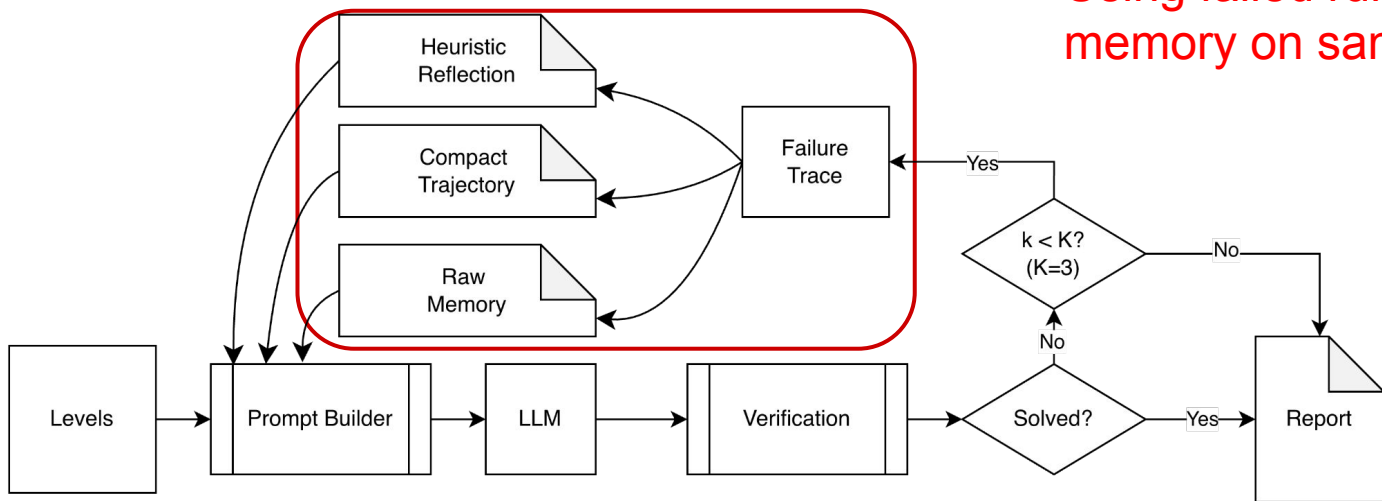
H1: With legal-push scaffolding and richer verifier feedback, memory should help action selection quality instead of being drowned by basic legality errors.

H2: Reflection memory should outperform raw trajectory memory on deadlock avoidance

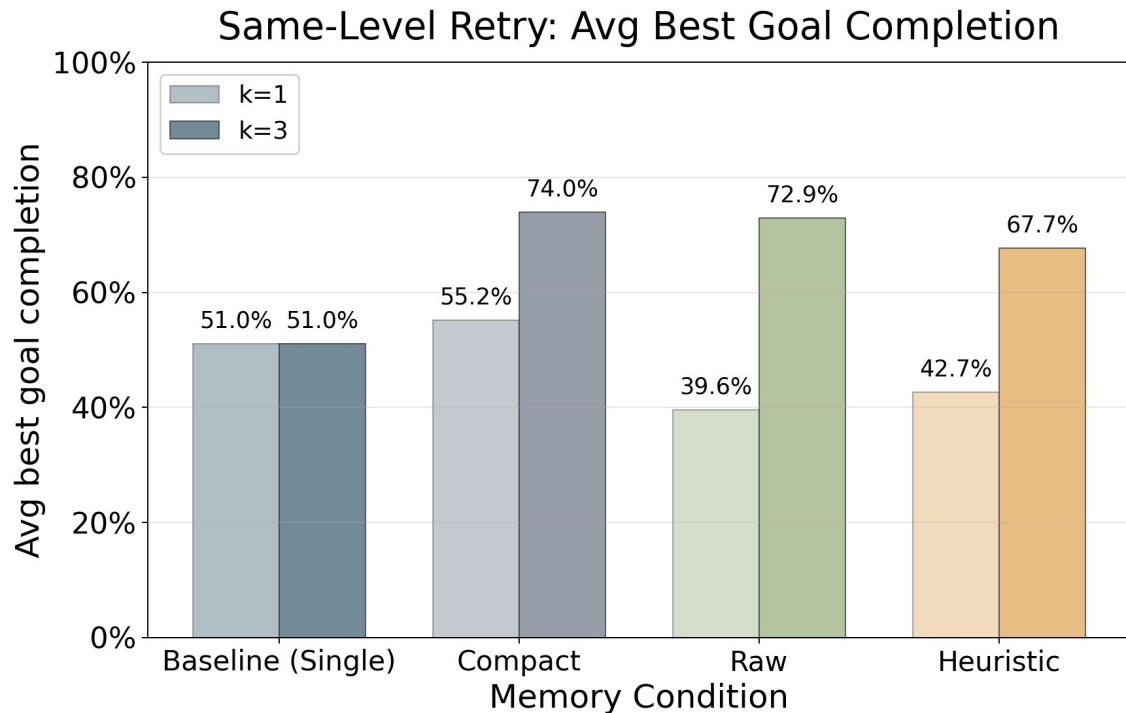
H3: Better harness quality is a prerequisite for fair memory comparison

Same Level Retry Results

Same-Level Memory Retry Pipeline



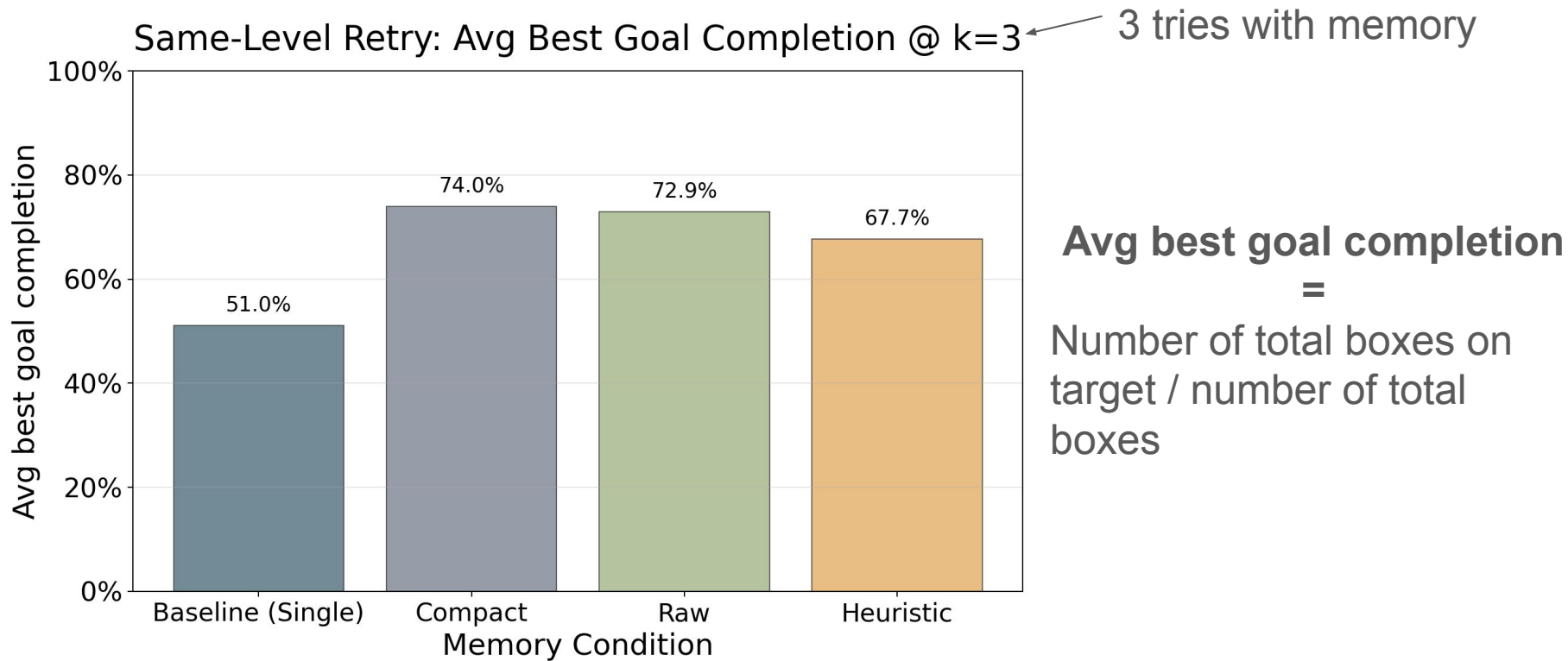
Results: Same-Level Retry Solve Rate Results



What is avg best goal completion?

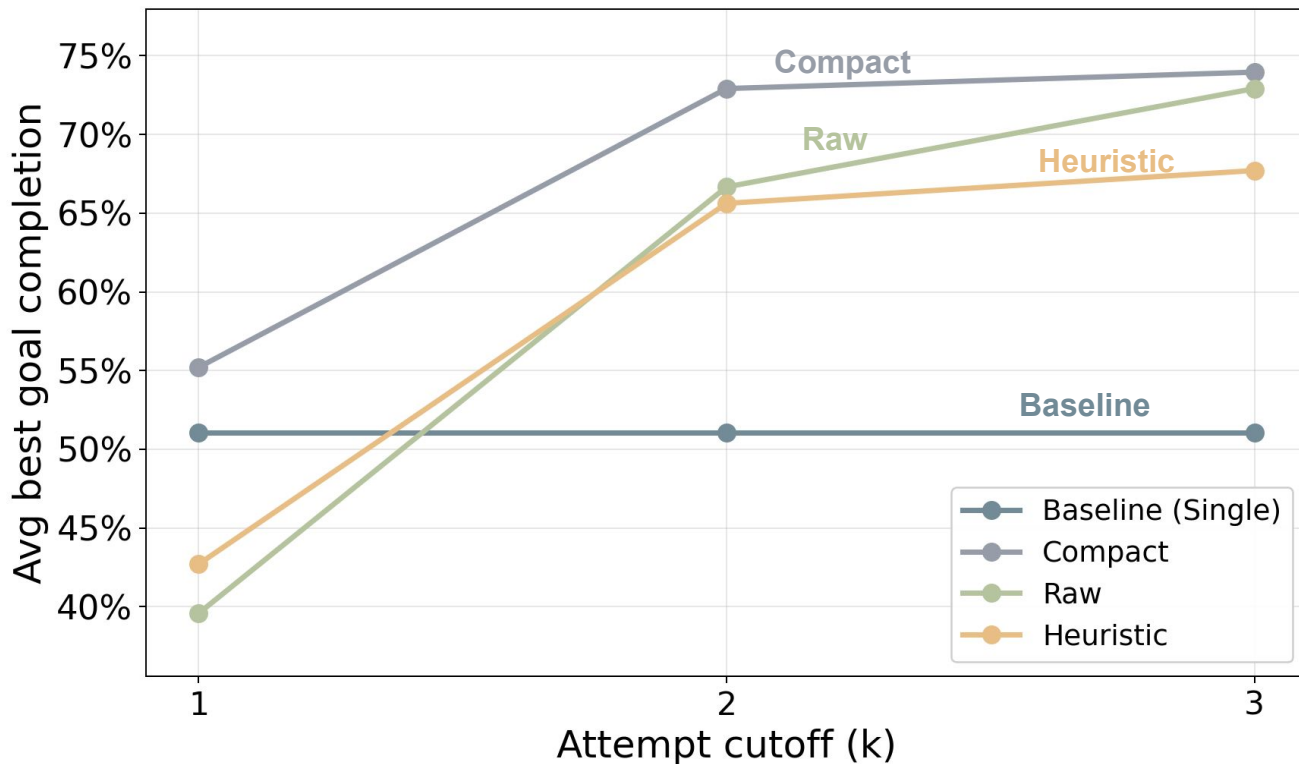
Maybe no k=1 as it is an artifact

Memory (Specifically Compact) Improves Partial Progress



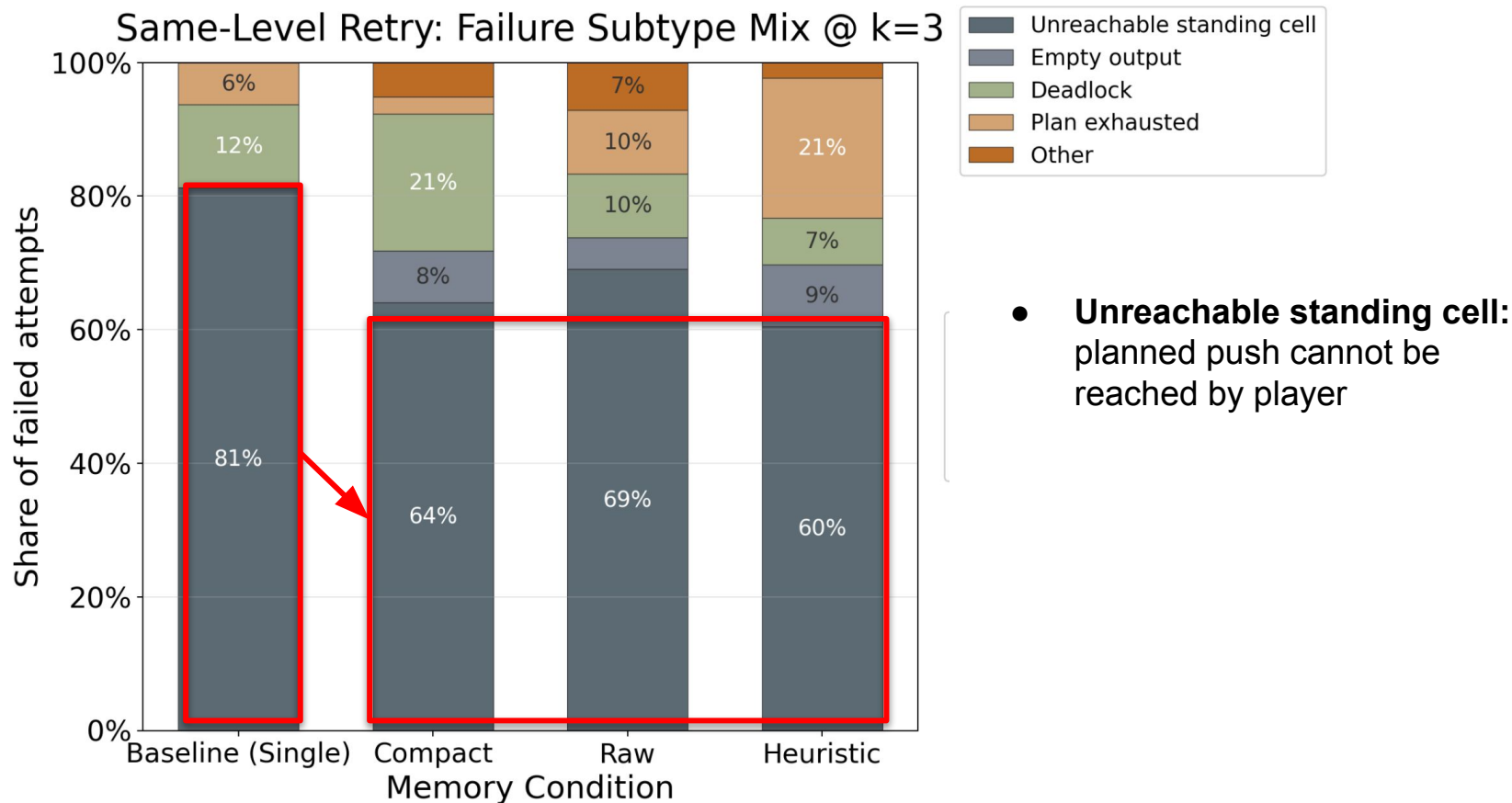
Results: cumulative solved-by-attempt curve

Same-Level Retry: Avg Best Goal Completion by Attempt

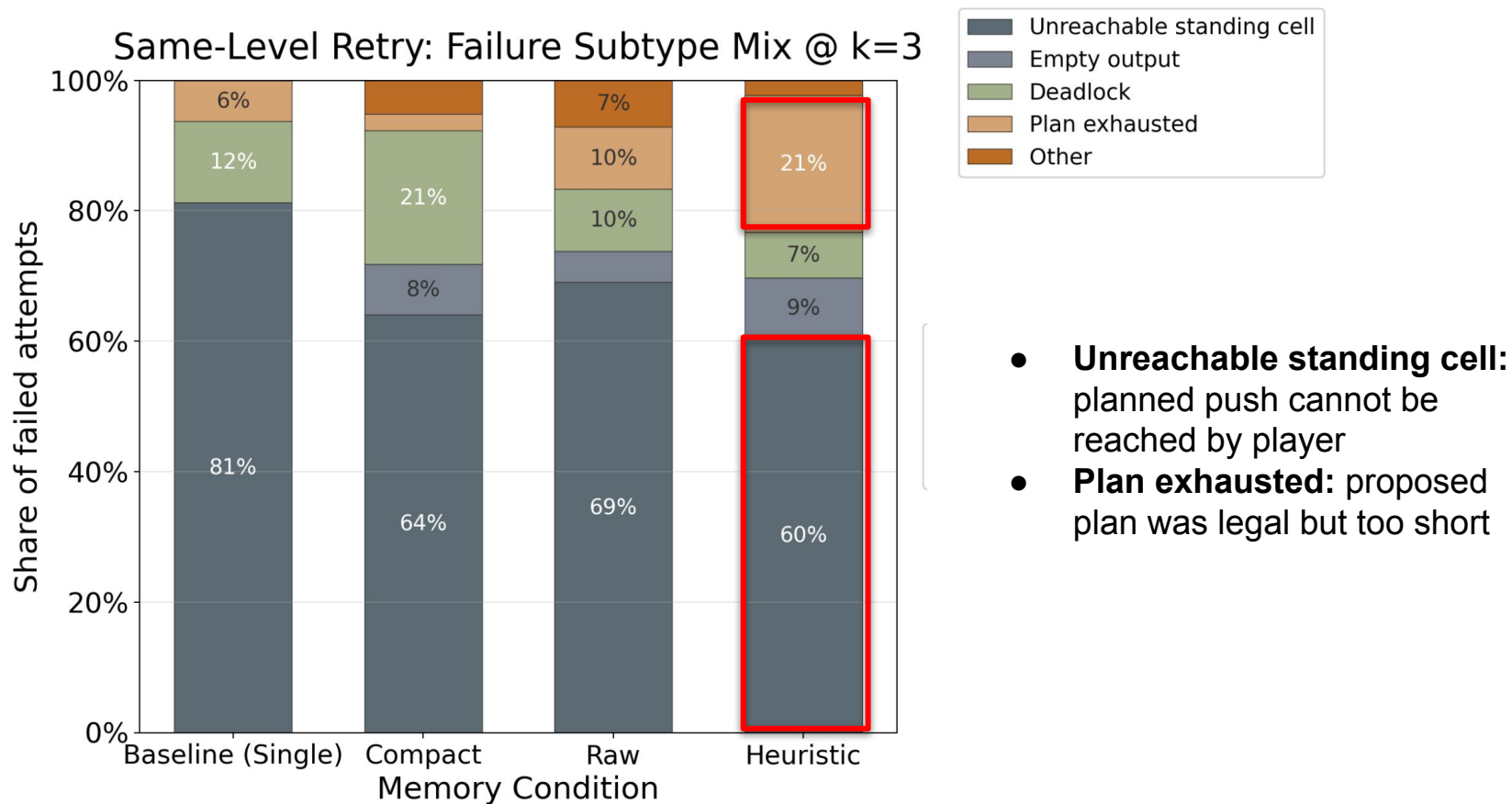


Why is
solve rate
at 1 so bad

Unreachable Standing Cell Failures Decrease with Memory



Heuristics Reduce Unreachable Pushes, But More Plans Run Out



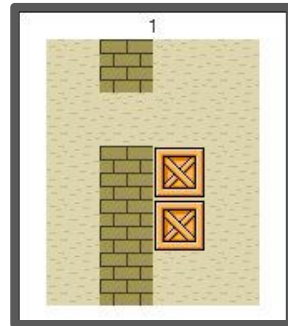
Heuristics improve general legality but not as much in specific gameplay improvements

Standing-cell reachability:

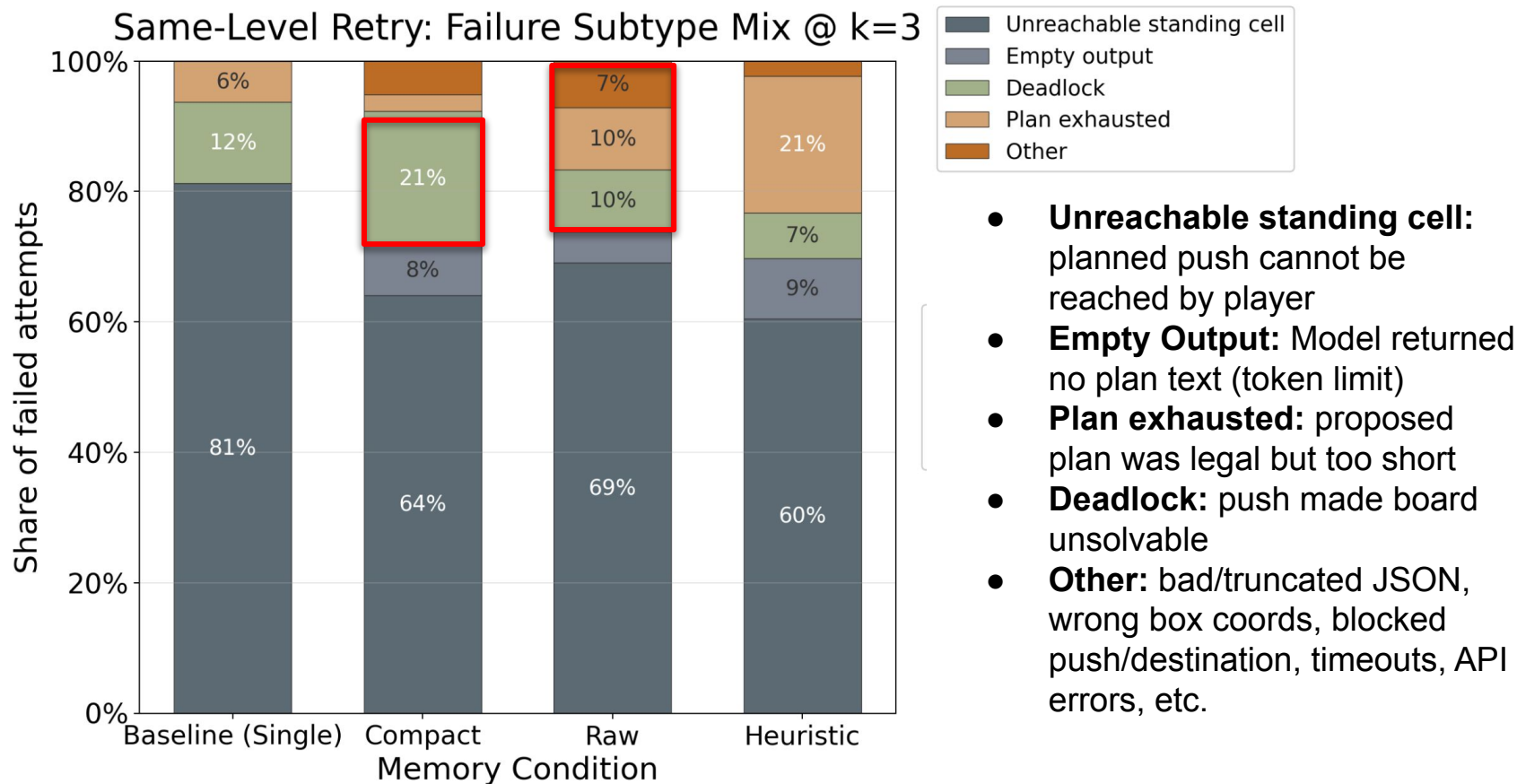
*“Before emitting a push intent, **verify the required standing cell** (one step opposite the push direction) is **reachable** from the player’s current region given the current box layout.”*

Corridor / chokepoint avoidance:

*“**Do not push** a box into a **1-tile-wide corridor** if that corridor is needed as a **transit route** to reach other pushes.”*



Compact Increases Deadlock Failures Compared to Raw



Overall, memory helps, but different memory representations improve different aspects of gameplay.

Compact Summary from the previous same-level attempt:

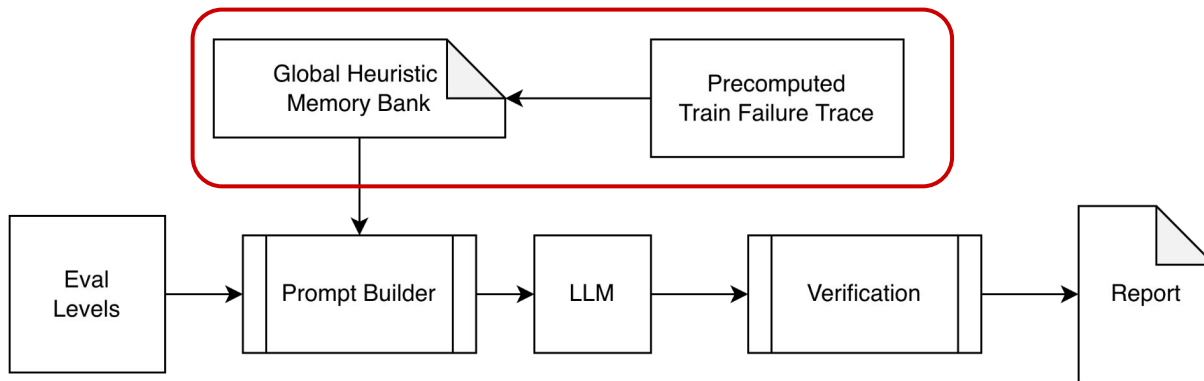
level_id: 1	Memory condition	Final outcome	Levels	Avg wall density	Avg reachable ratio	Avg initial legal pushes
failed_pushes	raw trajectory	Solved within K=3	13	0.676	0.535	7.00
failure_summary	raw trajectory	Failed all 3 attempts	11	0.655	0.461	5.00
verifier_retry	verifier_summary_retry	Solved within K=3	13	0.675	0.518	6.92
	verifier_summary_retry	Failed all 3 attempts	11	0.655	0.480	5.09
	heuristic	Solved within K=3	12	0.674	0.492	6.50
	heuristic	Failed all 3 attempts	12	0.658	0.510	5.67

Analysis + conclusions/findings

1. All three memory conditions beat the single-shot baseline @ K but `unreachable_standing_cell` failures dominates.
2. Instance-specific feedback beats one shot.
3. Verifier summary is the simpler signal; raw adds more context and sometimes rescues different levels
4. Heuristics improve general legality but not as much in specific gameplay improvements

Train to Eval Results

Train-to-Eval Heuristic Pipeline



Top 3 Heuristics

Standing-cell reachability (current state)

Before committing to a push intent, require that the player can reach the needed standing cell (the cell opposite the push direction from the box) in the current state; reject pushes whose standing cell is only reachable in the pre-push layout.

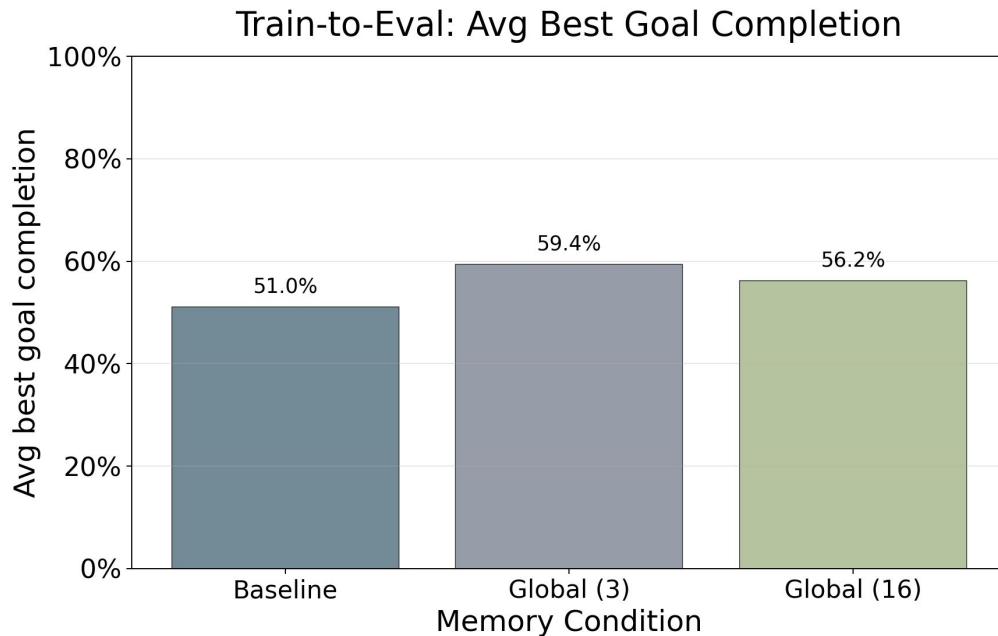
Corridor / connectivity fragility

Treat 1-tile corridors and chokepoints as fragile: avoid pushing any box into a corridor if it reduces the player's connectivity graph (creates a cut) unless that box is destined to stay there (goal or final storage).

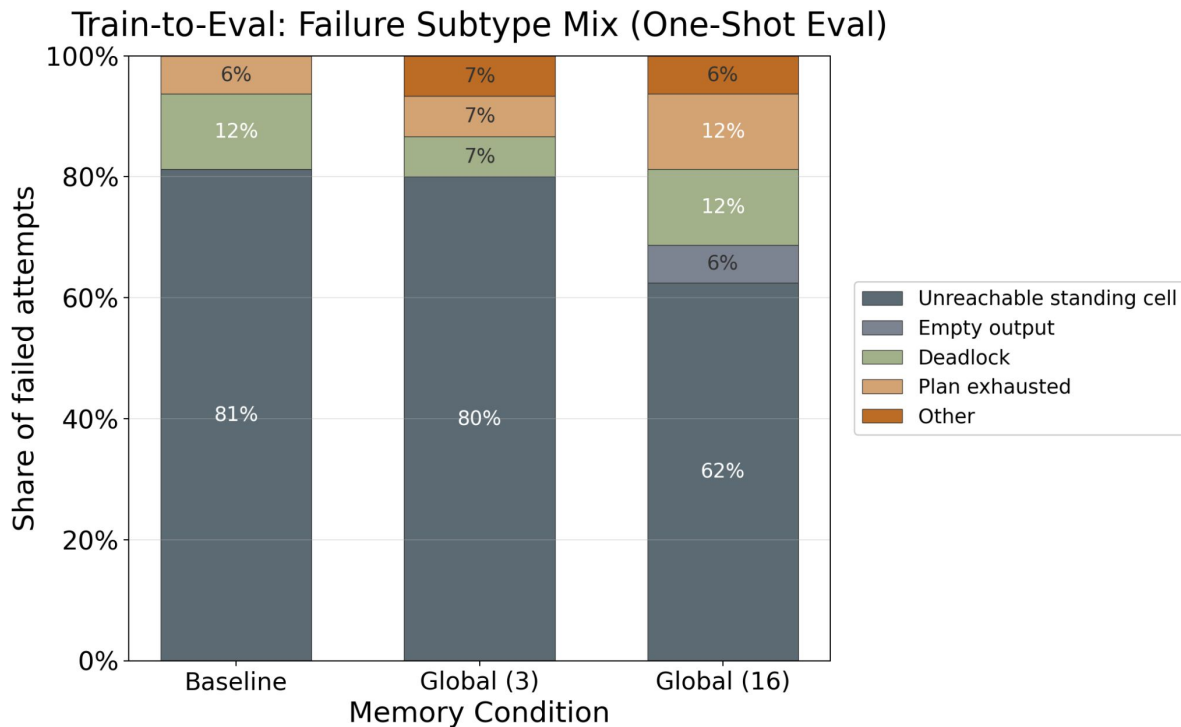
Self-blocking against walls

Never push a box to a wall-adjacent cell if the only way to push it again would require standing in a corner/against a wall that will become inaccessible after the push (classic self-blocking).

More Heuristics $\neq$ Better LLM Performance



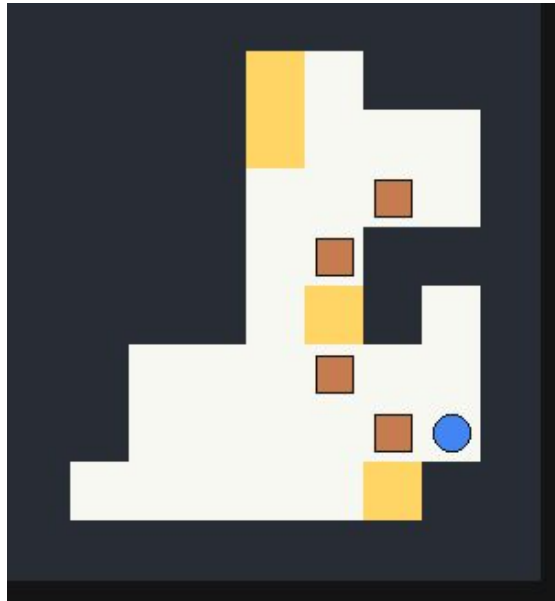
Unreachable Standing Cells Remain the Dominant Failure Type



Conclusion: Memory helps improve LLM Sokoban Gameplay But Instance-specific Feedback Helps Most

- Especially helpful in Same-level retry.
- Heuristics improve overall legality but not as helpful towards improving performance

Thank you!



Questions on results

1. Why is solve rate @ 1 so different
2. Why is heuristic worse at k runs → reusable and abstract
3. Why is it plateauing after 2 runs (for same level verifier)?
4. **In train to eval why no raw memory results**
5. Why are the number of attempts different
6. What can we learn from looking at which puzzles improved and which is failed on? (what did it learn from the memory). What does the memory say that would have allowed this?
7. Next step: comparing different methods of generating heuristics

Questions to answer??

- His Questions :

- What was the project being solved (defined in terms of goals/constraints, etc.)
- What specific technical challenge did you have to overcome? — how to create memory (token limit, for raw trajectory memory what part to pick out)? How to get solve rate, how to decide the prompt (full path vs single step, verification), how to choose and create levels, how to learn from failures
- What is the definition of success in your terms? – how many boxes pushed to target places without invalid/ (using verifier mechanism)
- What questions did you ask to determine if success was achieved? And what experiments did you perform to answer those questions?
- What do the results of those experiments suggest about the extent to which you achieved your definition of success?

Sub Questions:

- How good are LLMs initially at solving Sokoban?
- What kinds of failures do LLMs see in Sokoban?
- What are important heuristics for LLMs to solve Sokoban?

Question to Answer

- What was the project being solved (defined in terms of goals/constraints, etc.)
 - How good are LLMs initially at solving Sokoban?
 - What kinds of failures do LLMs see in Sokoban?
 - Does memory improve the performance of llm on Sokoban
- What specific technical challenge did you have to overcome?
 - how to create memory (token limit, for raw trajectory memory what part ot pick out)?
 - How to get solve rate, how decided the prompt (full path vs single step, verification),
 - how to choose and create levels,
 - how to learn from failures
- What is the definition of success in your terms?
 - Understanding and having evidence of how memory strategies can impact Sokoban gameplay,
 - Having a better understanding of how to choose/refine memory that helps
 - how many boxes pushed to target places without invalid/ (using verifier mechanism)
- What questions did you ask to determine if success was achieved? And what experiments did you perform to answer those questions?
- What do the results of those experiments suggest about the extent to which you achieved your definition of success?

- Levels
 - Create
 - How
 - Source

- Memory
 - Types
 - Create
 - Usage

- Play
 - LLM api

- Verification
 - how,
 - Failure type
 - Off